APPENDIX 1

A CONTEMPORARY TRANSPORT MANEGEMENT SYSTEM USING FLUTTER FRAMEWORK

A PROJECT REPORT

Submitted By

SURENDAR. A -421221104045

SWETHA. V -421221104046

TAMILSELVI. P -421221104047

SHAHINA. F -421221104035

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

KARPAGA VINAYAGA COLLEGE OF ENGINEERING AND TECHNOLOGY

ANNA UNIVERSITY: CHENNAI 600 025

MAY 2025

APPENDIX 2

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report **"A CONTEMPORARY TRANSPORT MANAGEMENT SYSTEM USING FLUTTER FRAMEWORK"** is the Bonafide work of **"SURENDAR. A (421221104045), SWETHA. V (421221104046), SHAHINA. F (421221104035), TAMILSELVI. P (421221104047)"** who carried out the project work under my supervision.

SIGNATURE	**SIGNATURE**
(Dr. C. Jayapradha) **PROFESSOR & HEAD OF THE DEPARTMENT** Computer Science and Engineering GST Road, Chinna Kolambakkam, Palayanoor PO, Madurantagam Taluk, Chengalpattu, Tamil Nadu 603 308.	(Dr. V.S. Thiyagarajan) **ASSOCIATE PROFESSOR & SUPERVISOR** Computer Science and Engineering GST Road, Chinna Kolambakkam, Palayanoor PO, Madurantagam Taluk, Chengalpattu, Tamil Nadu 603 308.

Submitted for the University Project Viva voice held on ……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

With profound gratitude and due regards, we whole heartedly and sincerely acknowledge with thanks the opportunity provided to us by our Respectful **Director Dr. Meenakshi Annamalai** for allowing us to do this project in partial fulfillment for the degree of Bachelor of Engineering under Anna University, Chennai.

We thank our dedicated **Principal Dr. P. Kasinatha Pandiyan** for his valuable suggestions and timely advice which helped us in completing this project on schedule.

Grateful to our esteemed **Dean Dr. L. Subbaraj** for your inspiring leadership and unwavering support. Your guidance has truly shaped our academic growth and future aspirations.

We thank our **Head of the Department Dr. C. Jayapratha, Ph.D.** for allotting us this project and her able guidance.

We thank our **Project Guide Dr. V.S. Thiyagarajan** for his painstaking efforts and proper guidance without which this project could not have been completed.

We thank our various faculty members and friends for their timely help and guidance in one way or other which went a long way in the completion of this project.

We will be failing in our duty if we don't thank our parents for their benevolence and blessings which stood us in good stead during the course of the project.

We would like to thank the authors of various journals and books whose works and results are used in this project.

A CONTEMPORARY TRANSPORT MANEGEMENT SYSTEM USING FLUTTER FRAMEWORK

ABSTRACT

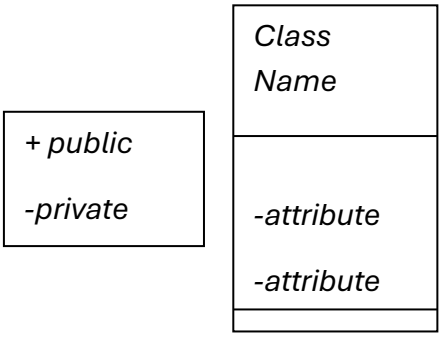
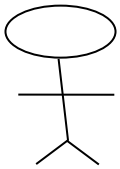
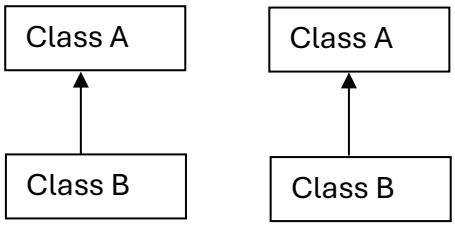
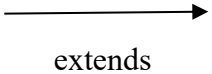
This project introduces a Flutter-based Transportation Management System (TMS) designed to enhance the efficiency and coordination of student transportation services. The system comprises three distinct modules—Student, Driver, and Admin—each tailored to meet the specific requirements of its users. The student module facilitates user registration and authentication, enabling students to select bus routes based on their current location, track real-time bus positions via Google Maps integration, and access delay reports submitted by drivers. The Driver module allows drivers to register, log in, update trip details including start times and current locations, submit delay reports, and manage student attendance by scanning student ID QR codes. The admin module provides administrators with comprehensive control over the system, allowing them to manage driver and bus information, student records, attendance logs, and delay reports, thereby ensuring effective oversight of transportation logistics. By integrating real-time tracking, QR code-based attendance, and a user-friendly interface, the proposed system aims to streamline transportation operations, enhance communication among stakeholders, and improve overall service reliability.

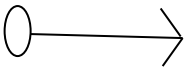
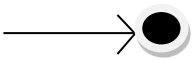
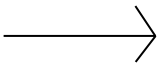
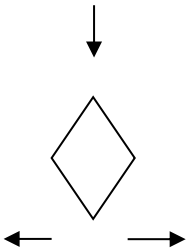
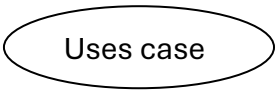
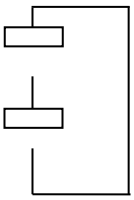
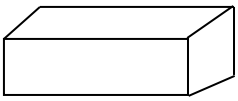
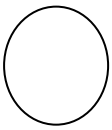
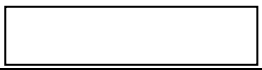
APPENDIX 3

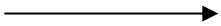
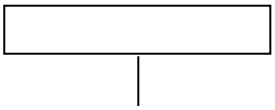
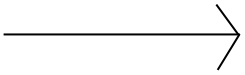
TABLE OF CONTENTS

LIST OF FIGURES

LIST OF SYMBOLS

S.NO	NOTATION NAME	NOTATION	DESCRIPTION
1.	Class		Represents a collection of similar entities grouped together.
2.	Association		Associations represent static relationships between classes. Roles represent the way the two classes see each other.
3.	Actor		It aggregates several classes into a single class.
4.	Aggregation		Interaction between the system and external environment.
5.	Relation (uses)	**Uses**	Used for additional process communication.
6.	Relation (extends)		Extends relationship is used when one use case is similar to another use case but does a bit more.
7.	Communication		Communication between various use cases.

8.	State		State of the process.
9.	Initial State		Initial state of the object
10.	Final State		Final state of the object
11.	Control Flow		Represents various control flow between the states.
12.	Decision Box		Represents decision making process from a constraint
13.	Use case		Interaction between the system and external environment.
14.	Component		Represents physical modules which is a collection of components.
15.	Node		Represents physical modules which are a collection of components.
16.	Data Process / Sate		A circle in DFD represents a state or process which has been triggered due to some event or action.
17.	External entity		Represents external entities such as

			keyboard, sensors, etc.
18.	Transition		Represents communication that occurs between processes.
19.	Object Lifeline		Represents the vertical dimensions that the object communications.
20.	Message		Represents the message exchanged.

LIST OF ABBREVATIONS

S.NO	ABBREVATION	EXPANSION
1.	DB	Data Base
2.	SMC	Secure Multiparty Computation
3.	SC	Service Center
4.	DBC	Data Base Confidentiality
5.	JVM	Java Virtual Machine
6.	JSP	Java Server Page

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

Efficient transportation systems are crucial for ensuring student safety and punctuality while facilitating effective communication among stakeholders. Traditional methods often rely on manual processes, leading to inefficiencies and a lack of real-time information. This project presents a Flutter-based Transportation Management System (TMS) designed to enhance the coordination of transportation services for students, drivers, and administrators. The system comprises three modules: Student, Driver, and Admin, each tailored to specific user roles. The student module allows users to register, select bus routes based on their location, track buses in real-time via Google Maps, and view delay reports submitted by drivers. The Driver module enables drivers to manage trip details, update current locations, submit delay reports, and record student attendance using QR code scanning. The admin module provides a centralized interface for managing drivers, buses, students, attendance logs, and delay reports, ensuring organized transportation oversight. By leveraging real-time updates and a user-friendly design, the system aims to improve communication, transparency, and operational effectiveness in the transportation process.

1.2 EXISTING SYSTEM

The current transportation management systems heavily rely on accurate data about transportation infrastructure to ensure effective route planning. However, the rapid development of new transit lines and the establishment of temporary bus stops often lead to discrepancies between actual conditions and the data provided by mapping applications. These inconsistencies can result in passenger confusion, service delays, and increased complaints directed at public transportation operators. Addressing these challenges is complex due to the dynamic nature of urban development and the difficulties in maintaining real-time data accuracy.

Techniques

GIS FRAMEWORK

1.3 PROPOSED SYSTEM

The proposed project introduces a Flutter-based transportation management system designed to streamline operations for students, drivers, and administrators. The student module allows registration, bus travel with location filtering, real-time tracking, and late form viewing. The Driver module includes features for updating trip details, sharing location, submitting late forms, and scanning student attendance via QR codes. The admin module provides a centralized platform for managing and viewing information about drivers, buses, students, late forms, and attendance, improving overall system efficiency and communication.

Techniques

Realtime database, MOBILE GPS TRACKING SYSTEM, Firebase, Flutter, GOOGLE MAP API

1.3.1 OBJECTIVES

The primary objective of this project is to design and implement a comprehensive, Flutter-based Transportation Management System (TMS) that addresses the complexities of modern student transportation services. The system aims to provide a centralized platform that enhances communication, coordination, and operational efficiency among students, drivers, and administrators.

Real-Time Bus Tracking: Integrating GPS technology to allow students and administrators to monitor bus locations in real-time, thereby improving schedule adherence and reducing waiting times.

Digital Attendance Management: Implementing QR code scanning for drivers to record student attendance, ensuring accurate and efficient tracking of student ridership.

Automated Late Form Submissions: Enabling drivers to submit delay reports digitally, streamlining the process and providing administrators with timely information on service disruptions.

Comprehensive Data Management: Developing modules for administrators to manage and analyse data related to routes, users, schedules, and attendance, facilitating informed decision-making and operational oversight.

1.3.2 SCOPE

This project encompasses the design and development of comprehensive, Flutter-based Transportation Management System (TMS) tailored to meet the needs of educational institutions. The system is structured into three primary modules—**Student**, **Driver**, and **Admin**—each serving distinct user roles to facilitate efficient and seamless transportation operations.

Student Module

Enables students to register, log in, select bus routes based on their current location, track buses in real-time via GPS integration, and view late form submissions by drivers.

Driver Module

Allows drivers to register, log in, update trip details, share live locations, submit late forms, and record student attendance through QR code scanning.

Admin Module

Provides administrators with a centralized platform to manage and view information about drivers, buses, students, attendance records, and late forms, ensuring effective oversight and streamlined operations.

1.4 LITERATURE SURVEY

TITLE: A Bus Planning Algorithm for FPC Design in Complex Scenarios

AUTHOR: Haoying WU1, Sizhan ZOU1, Ning XU1, Shixu XIANG1, and Mingyu LIU,

YEAR: 2024

DESCRIPTION

[1] The design of Flexible Printed Circuits (FPCs) in complex scenarios often encounters challenges due to areas with high pin concentration, leading to congested intersection regions during routing. Currently, manually exploring rout ability and identifying topologically non-crossing paths for nets in a timely manner remains difficult. Existing bus planning methods fall short in providing optimal solutions that account for the unique resource distribution inherent in FPC designs. To address these challenges and enhance performance, this study proposes a bus planning algorithm tailored to complex area connection problems. By utilizing pin location information, the routing space is partitioned and represented as an undirected graph, with topological non-crossing relationships determined through dynamic pin sequencing. Considering both rout ability and electrical constraints, a heuristic algorithm is introduced to identify optimal crossing points along region boundaries. Experimental results on industrial cases demonstrate that the proposed algorithm outperforms several state-of-the-art routers in terms of count and rout ability.

TITLE: Topic Modelling on Document Networks With Dirichlet Optimal Transport Barycentre

AUTHOR: Delvin Ce Zhang, Hady W. Lauw

YEAR: 2024

DESCRIPTION

[2] Textual documents are often interconnected within a network structure, such as academic papers linked via citations or web pages connected through hyperlinks. While Graph Neural Networks (GNNs) have demonstrated promising capabilities in deriving effective embeddings for these networked documents, they typically do not assume an underlying latent topic structure, resulting in embeddings that lack interpretability. Conversely, topic models can infer semantically interpretable topic distributions by associating each topic with a

group of understandable keywords. However, most traditional topic models primarily focus on the plain text within documents and fail to leverage the network structure across documents. The connectivity between documents reveals topic similarities, and modelling this connectivity could uncover meaningful semantics. Motivated by these challenges, we propose a GNN-based neural topic model that captures network connectivity and derives semantically interpretable topic distributions for networked documents. For modelling network structure, our approach allows the topic distribution of a document to influence the content of its linked neighbours. To enhance semantic interpretability, we incorporate Dirichlet distributions as priors, facilitating the identification of coherent topics. Additionally, we employ optimal transport theory to align topic distributions across documents, ensuring consistency and interpretability. Extensive experiments on document classification, clustering, link prediction, and topic analysis demonstrate the effectiveness of our model. Our approach outperforms several state-of-the-art methods, highlighting its potential for capturing both semantic and structural information in networked documents.

TITLE: Bus Admittance Matrix Revisited: Performance Challenges on Modern Computers

AUTHOR: HANTAO CUI

YEAR: 2024

DESCRIPTION

[3] The bus admittance matrix (Y-bus) is a fundamental tool in power engineering for network modelling, particularly in power flow analysis. Its sparse nature allows for efficient computations, requiring fewer CPU operations. However, sparse matrix calculations involve numerous indexing and scalar operations, which can be less favourable for modern processors. An alternative approach involves computing nodal power injections and the corresponding sparse Jacobian matrix using an element-wise method. This method employs a highly regular, vectorized evaluation step followed by a reduction step, which can be more efficient on modern hardware. This paper revisits the computational performance of the Y-bus method by comparing it with the element-wise method in terms of power injection and Jacobian matrix calculation. Case studies demonstrate that the Y-bus method is generally slower than the element-wise method for grid test cases with thousands to hundreds of thousands of buses,

especially on CPUs with support for wide vector instructions. The analysis also considers the impact of vector instruction width and memory speed, providing insights into the performance trends for future computing architectures.

TITLE: Smart Public Transport System Using Mobile Application with Flutter Framework
AUTHOR: PRIYA S., ARUN KUMAR M., and DIVYA R.
YEAR: 2023

DESCRIPTION:

[4] The increasing demand for intelligent and user-friendly public transport systems has led to the integration of mobile technologies for real-time tracking, scheduling, and passenger interaction. Flutter, Google's open-source UI toolkit, enables rapid development of natively compiled mobile applications for both Android and iOS platforms from a single codebase. This paper presents the design and implementation of a smart transport management system developed using Flutter. The system allows users to check bus/train schedules, track vehicle locations in real-time using GPS, receive delay notifications, and book seats through a single application. The use of Flutter reduces development time and cost, while ensuring consistent performance across devices. The backend services are integrated via Firebase and REST APIs, ensuring secure and efficient data handling. Case studies show that the application improves user satisfaction and system reliability, with quick response times and smooth UI performance on mid-range smartphones. The paper also evaluates the application's scalability, maintainability, and performance across various devices and network conditions.

CHAPTER 2
PROJECT DESCRIPTION

2.1 GENERAL

2.2 METHODOLOGIES

Methodologies is the process of analysing the principles or procedure of a Progressive Anonymous Database management system.

2.2.1 MODULES NAME

- DRIVER
- STUDENT
- ADMIN

2.2.2 MODULE DESCRIPTION

1. DRIVER

The Driver module within the Flutter-Based Transportation Management System is meticulously designed to empower drivers with efficient tools for managing their daily operations. Upon registration and login, drivers gain access to a suite of functionalities that streamline their tasks and enhance communication with both students and administrators.

2. STUDENT

The student module in the Flutter-based Transportation Management System is designed to provide students with a convenient and interactive platform for managing their daily transportation needs. [5] Students can register, log in, and choose appropriate bus routes based on their current location. The module includes real-time bus tracking via Google Maps, enabling students to monitor the location of their assigned buses for timely pickups. Additionally, students can view delay notifications submitted by drivers to stay informed about any potential delays. This module enhances the user experience by offering easy access to transportation information, improving safety, and reducing uncertainty in daily commutes.

3. ADMIN

The admin module in the Flutter-based Transportation Management System serves as the central control unit for overseeing and managing all aspects of the transportation network. Administrators can log in to access and manage detailed records of students, drivers, buses, attendance, and late form submissions. This module provides tools to monitor real-time operations, verify driver and student information, review trip and delay reports, and ensure the smooth functioning of transportation services. By offering a comprehensive overview and control of system activities, the admin module enhances decision-making, improves operational efficiency, and ensures accountability across all user modules.

2.2.3 MODULE DIAGRAM

2.2.3.1 STUDENT

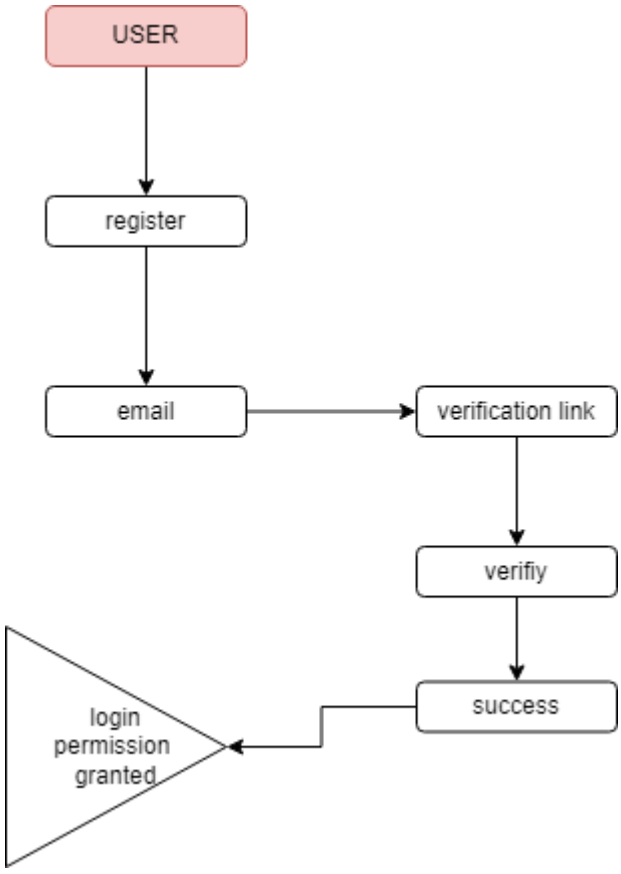


Fig.1.Student Module

2.2.3.2 DRIVER

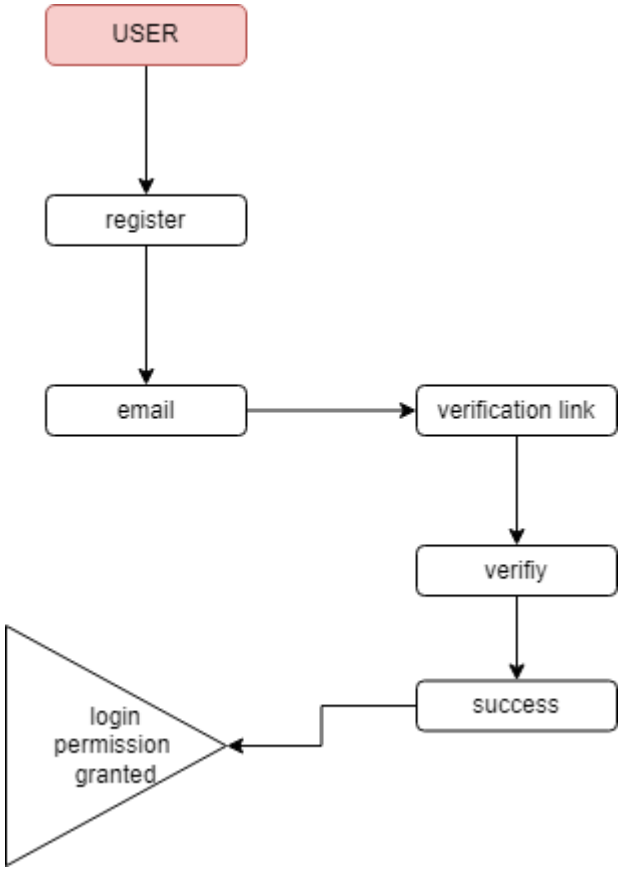


Fig.2.Driver Module

2.2.3.3 ADMIN

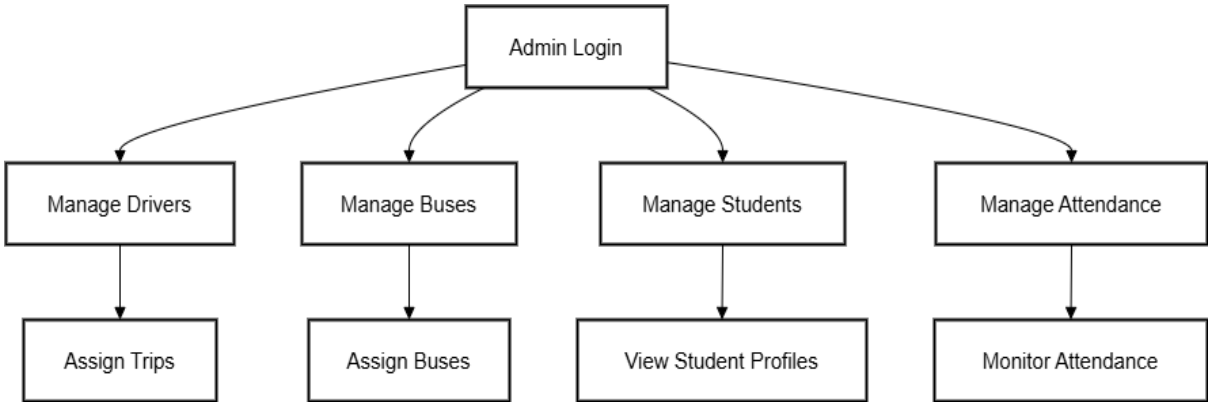


Fig.3.Admin Module

CHAPTER 3

REQUIREMENTS ENGINEERING

3.1 GENERAL

These are the requirements for doing the project. Without using these tools and software's we can't do the project. So, we have two requirements to do the project. They are

1. Hardware Requirements.
2. Software Requirements.

3.2 HARDWARE REQUIREMENTS

- PROCESSOR : GREATER THAN 13
- RAM : 4 GB DD RAM
- HARD DISK : 1 TB

3.3 SOFTWARE REQUIREMENTS

- FRONT END : FLUTTER
- OPERATING SYSTEM : WINDOWS 11
- IDE : ANDROID STUDIO
- DATABASE : FIREBASE

CHAPTER 4

DESIGN

4.1 GENERAL

Design Engineering deals with the various UML [Unified Modelling language] diagrams for the implementation of project. Design is a meaningful engineering representation of a thing that is to be built. Software design is a process through which the requirements are translated into representation of the software. Design is the place where quality is rendered in software engineering. Design is the means to accurately translate customer requirements into finished product.

4.1.1 USECASE DIAGRAM

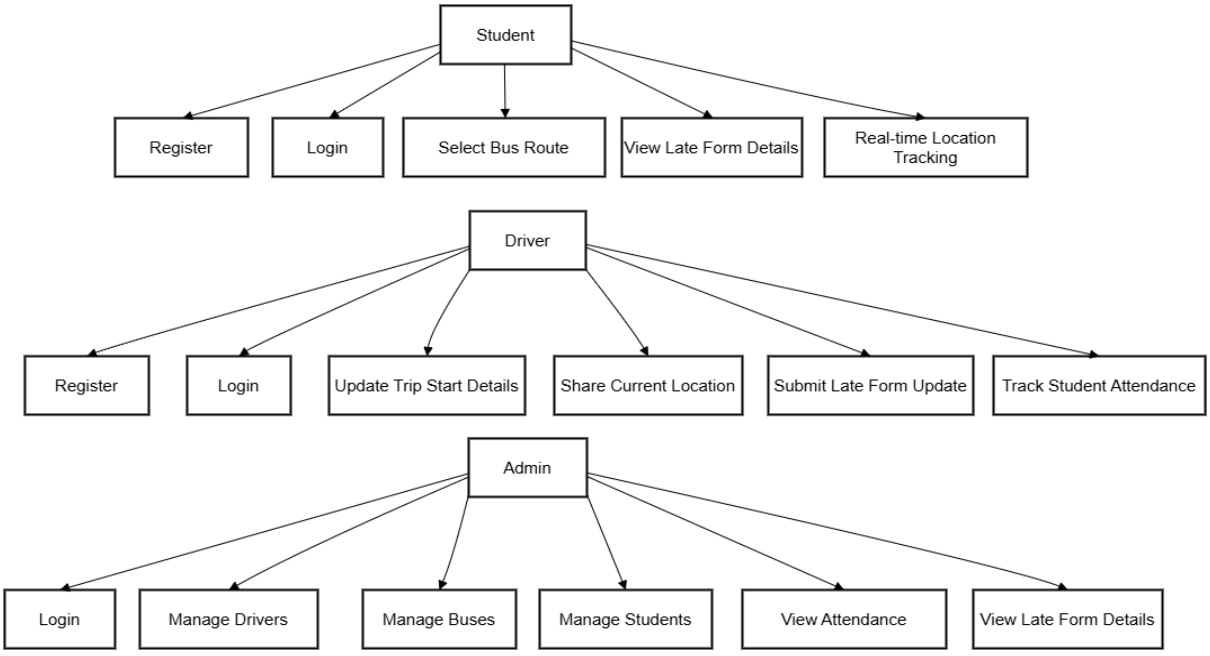


Fig.4.UseCase Diagram

DESCRIPTION

The **use case diagram** for the Flutter-Based Transportation Management System illustrates the interactions between the three main actors—**Student**, **Driver**, and **Admin**—and the system's core functionalities. Students can register, log in, select bus routes, track buses in real time, and view late form updates.[6] Drivers interact with the system by registering, logging in, starting trips, sharing live

location data, submitting late forms, and scanning student QR codes for attendance tracking. Admins oversee the entire system, managing driver and student records, bus details, attendance logs, and reviewing submitted late forms. The diagram effectively highlights how each user role interacts with specific features, ensuring a clear understanding of system flow and responsibilities among stakeholders.

4.1.2 STATE DIAGRAM

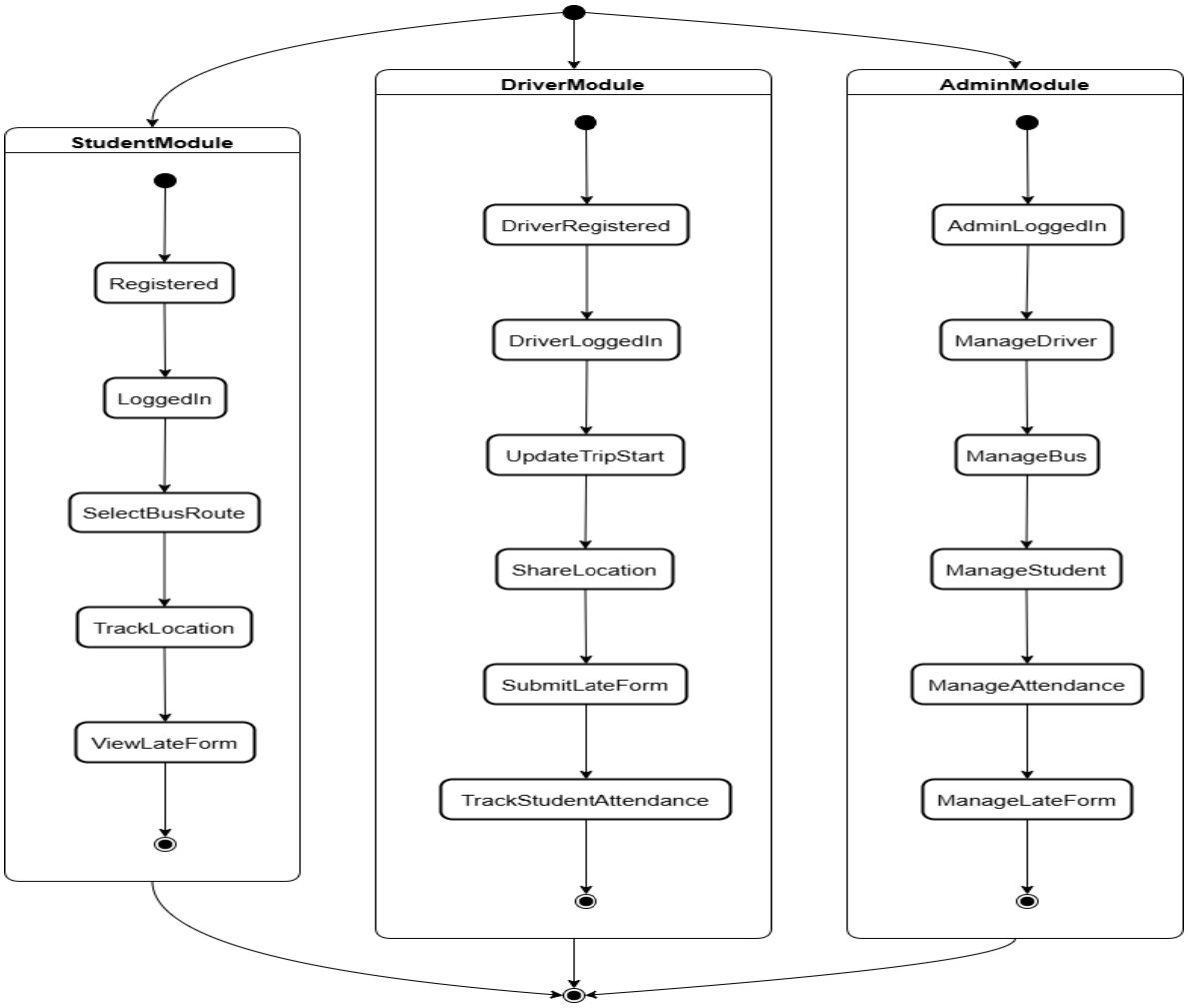


Fig.5.State Diagram

DESCRIPTION

The state diagram for the Flutter-Based Transportation Management System illustrates the dynamic behaviour of each user module—Student, Driver, and Admin—by depicting the various states and transitions triggered by user actions.

In the **Student module**, the states begin with Registration/Login, followed by Dashboard Access, where students can choose bus routes, enter the Bus Tracking state to view real-time locations via Google Maps, and transition to the View Late Forms state if delays are reported. [7] In the **Driver module**, after Login, the driver can enter states like Start Trip, Update Location, Submit Late Form, and Scan Student QR Codes for attendance. The **admin module** transitions from Login to Admin Dashboard, where the admin can access states like Manage Drivers, Manage Students, View Attendance Records, and Review Late Forms. Each module's states are interconnected through user inputs or system events, providing a clear flow of actions and ensuring responsive and interactive transport management across all stakeholders.

4.1.3 ACTIVITY DIAGRAM

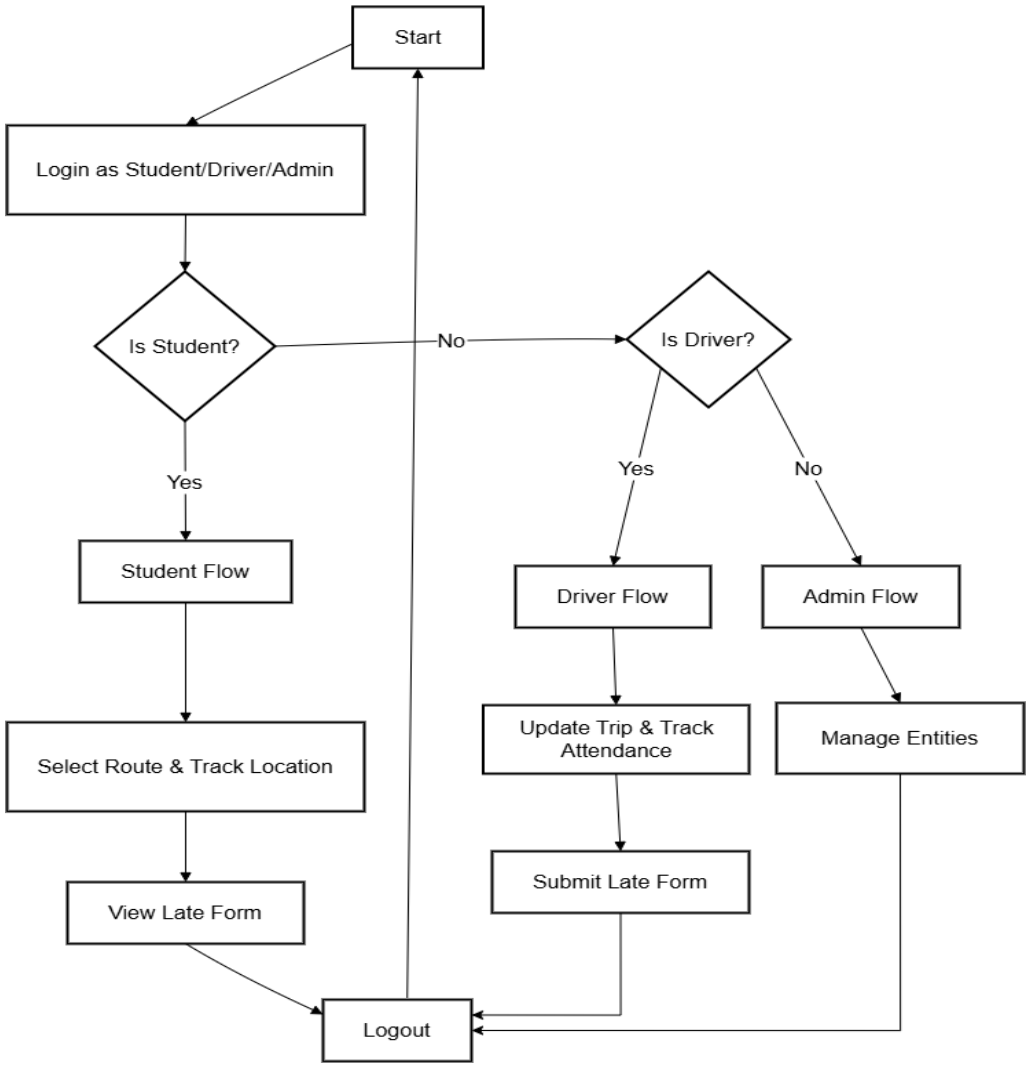


Fig.6.Activity Diagram

DESCRIPTION

The activity diagram for the Flutter-Based Transportation Management System outlines the sequence of actions performed by Students, Drivers, and Admins within the system. For **Students**, the activity starts with Login/Registration, followed by accessing the Dashboard, where they can Select Bus Route, Track Bus Location in real-time, and View Late Form Details if available. In the **Driver** workflow, the activity begins with Login, followed by actions like Start Trip, Update Current Location, Submit Late Form in case of delays, and Scan Student QR Code to mark attendance. The **admin** activity begins with Login, leading to a set of management functions such as View and Manage Student Records, Manage Driver and Bus Details, Monitor Attendance Logs, and Review Late Forms. The diagram flows logically from one action to the next using decision points and parallel actions where necessary, representing the collaborative and real-time nature of the system's transportation operations.

4.1.4 CLASS DIAGRAM

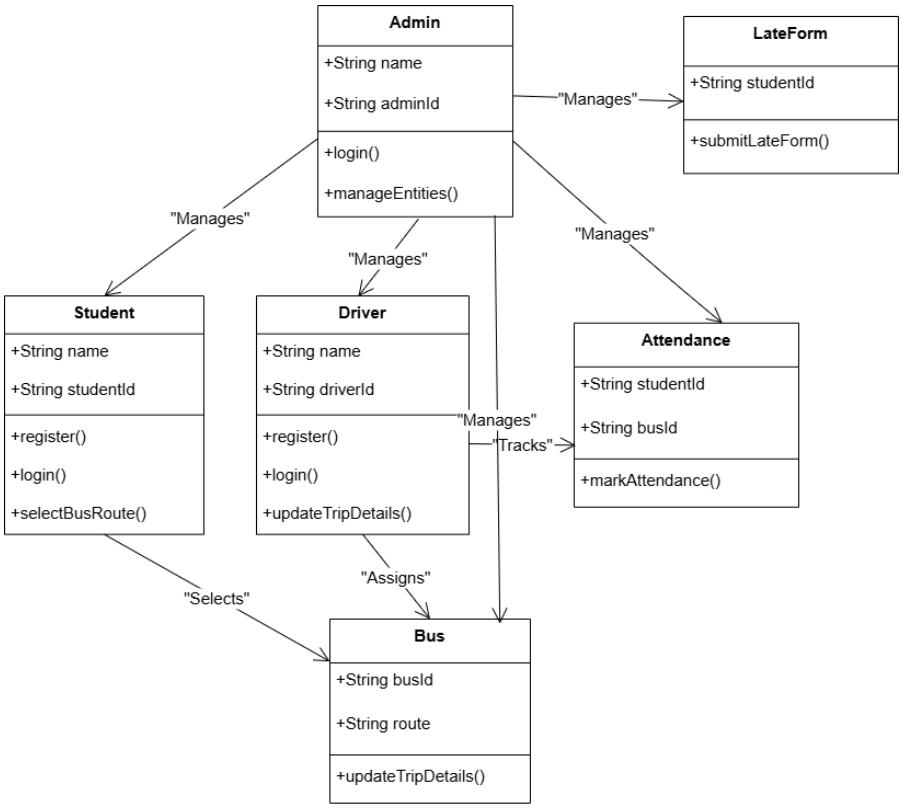


Fig.7.Class Diagram

DESCRIPTION

The class diagram for the Flutter-Based Transportation Management System represents the structural relationships among core entities across the Student, Driver, and Admin modules. The **Student** class includes attributes like studentID, name, email, route, and methods such as register(), login(), trackBus(), and viewLateForm(). The **Driver** class contains attributes such as driverID, name, busNumber, currentLocation, and includes methods like login(), startTrip(), updateLocation(), submitLateForm(), and scanQRCode(). The **Admin** class manages system-wide operations with attributes like adminID, name, and methods including login(), manageStudents(), manageDrivers(), viewAttendance(), and reviewLateForms(). Supporting classes like **Bus**, **Route**, **Attendance**, and **LateForm** are associated with these core classes, ensuring encapsulation and clear relationship mapping—such as one-to-many relations between Bus and Students, or between Driver and LateForm. This structure promotes modularity, reusability, and maintainability of the system.

4.1.5 DATAFLOW DIAGRAM

4.1.5.1 DESCRIPTION

The Data Flow Diagram (DFD) for the Flutter-Based Transportation Management System illustrates the flow of data between external users (Students, Drivers, and Admins), system processes, and data stores. At the top level, Students interact with the system to register/login, select bus routes, track real-time bus locations, and view late form updates, with data flowing to and from the **Student Database** and **Bus Location System**. Drivers input trip start details, update live location, scan student QR codes for attendance, and submit late forms, all of which interact with **Driver**, **Attendance**, and **Late Form Databases**. Admins manage and retrieve information about students, drivers, buses, and attendance by interacting with all key data stores through centralized admin processes like Manage Users, Monitor Attendance, and Review Reports. The DFD highlights how real-time data like location and attendance flows through processes, ensuring consistent updates and accessibility for all stakeholders, ultimately enhancing coordination and operational efficiency in the transportation system.

4.1.5.2 STUDENT

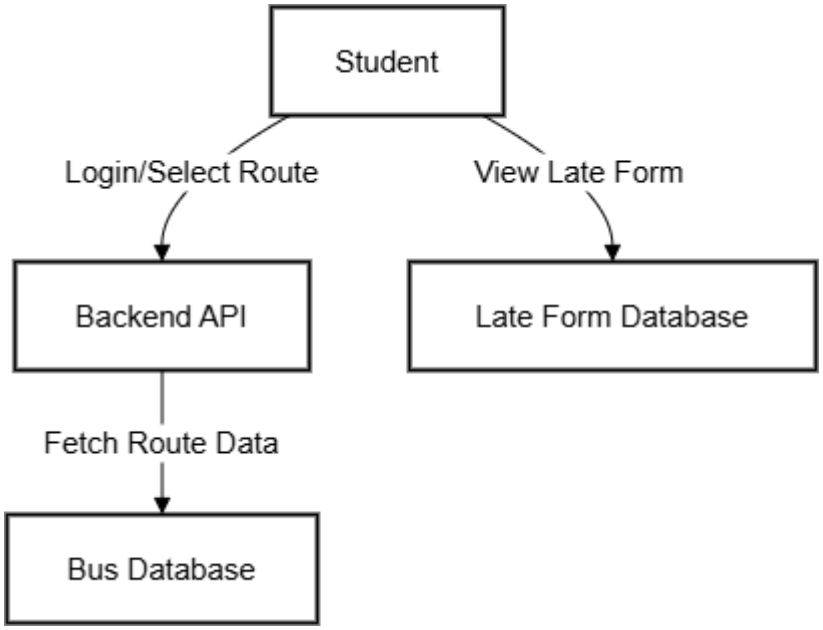


Fig.8.DataFlow Diagram of the Student

4.1.5.3 DRIVER

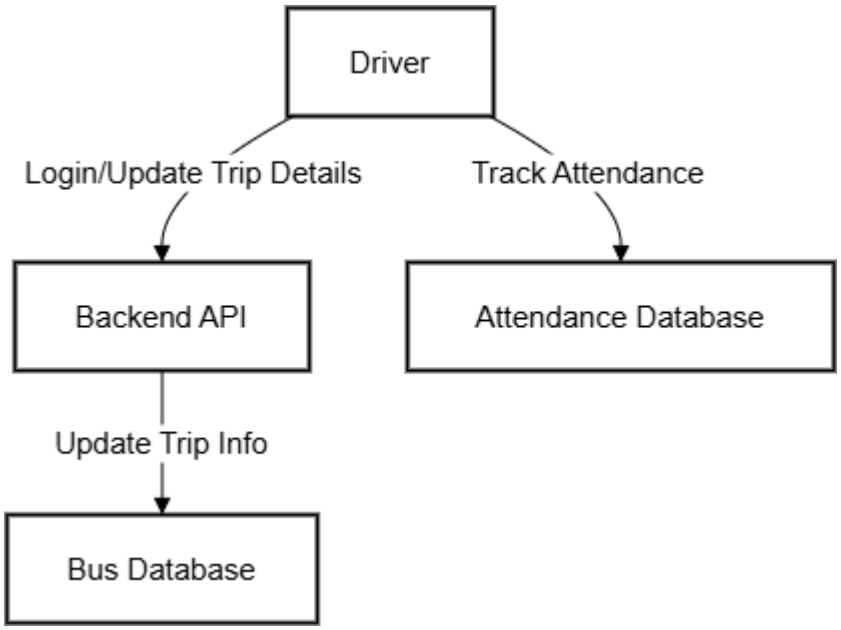


Fig.9.DataFlow Diagram of the Driver

4.1.5.4 ADMIN

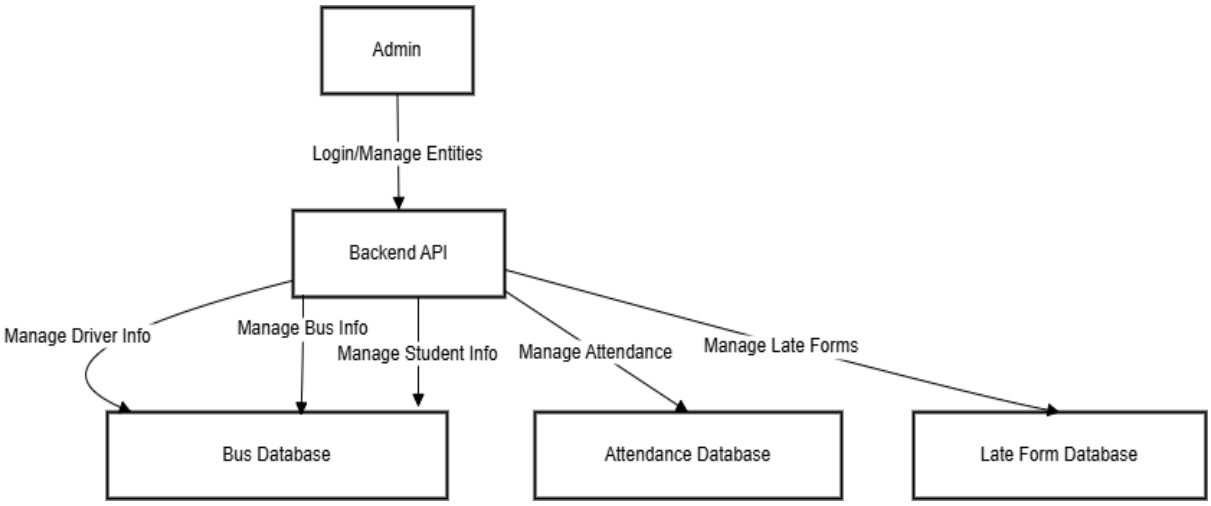


Fig.10.DataFlow Diagram of the Admin

4.1.6 E-R DIAGRAM:

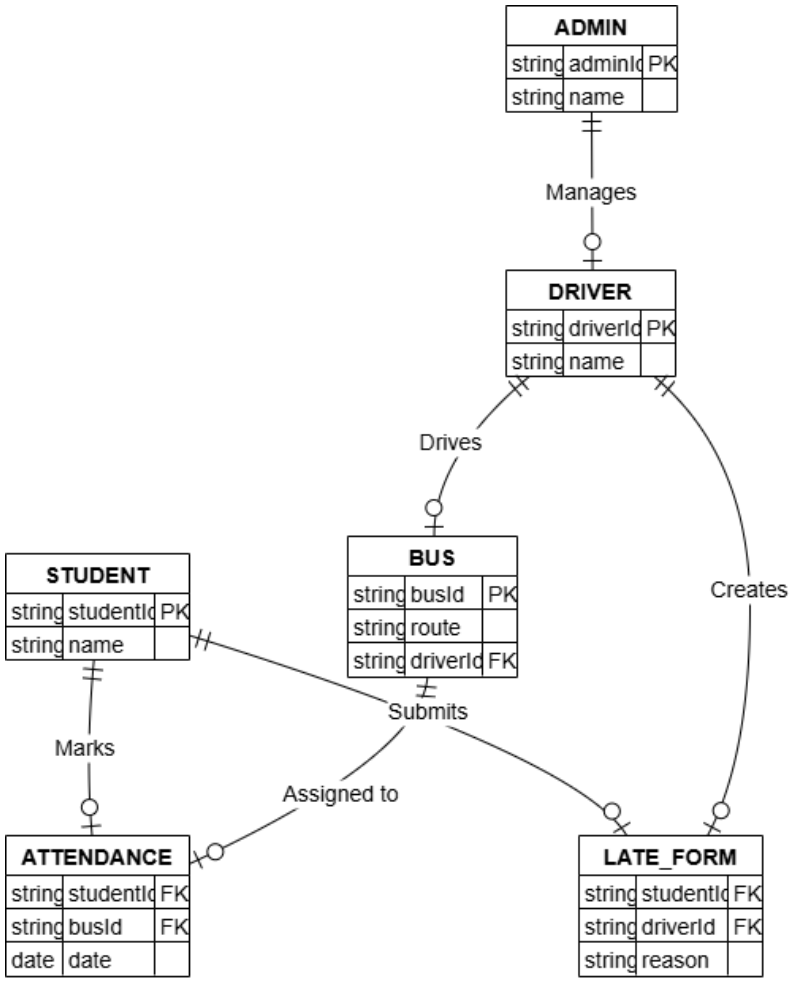


Fig.11.ER Diagram

DESCRIPTION

The Entity-Relationship (ER) diagram for the Flutter-Based Transportation Management System defines the key entities and their relationships across the Student, Driver, and Admin modules. The main entities include **Student**, **Driver**, **Admin**, **Bus**, **Route**, **Attendance**, and **LateForm**. The **Student** entity has attributes like studentID, name, email, and is related to the **Route** entity through a many-to-one relationship, indicating that many students can be assigned to one route. The **Driver** entity, with attributes such as driverID, name, and busNumber, is linked to the **Bus** entity in a one-to-one relationship. **Drivers** also have a one-to-many relationship with both **LateForm** and **Attendance**, as each driver can submit multiple late reports and record attendance for many students. The **Admin** entity oversees all other entities but does not maintain a direct relationship—rather, it acts as a controller managing data operations. This ER model establishes the logical structure of the database, ensuring referential integrity and efficient access to transportation-related data.

4.1.7 SYSTEM ARCHITECTURE

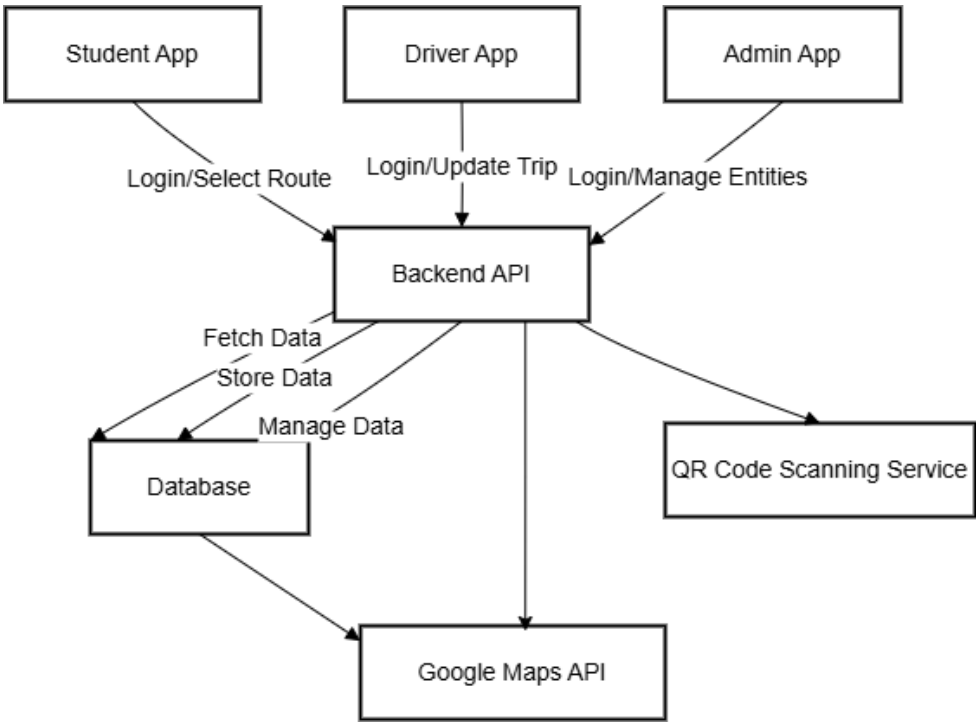


Fig.12.System Architecture

DESCRIPTION

The system architecture diagram for the Flutter-Based Transportation Management System showcases a three-tier architecture comprising the **Presentation Layer (Client Side)**, **Application Layer (Backend Logic)**, and **Data Layer (Database Services)**. The **Presentation Layer** includes the Flutter mobile interfaces for Students, Drivers, and Admins, allowing users to interact with the system through features like registration, login, bus tracking, QR scanning, and data viewing. These user actions are processed by the **Application Layer**, which handles authentication, business logic (such as route assignment, location tracking, and attendance management), and communication between modules via APIs. This layer also integrates with external services like Google Maps for real-time tracking. [8]The **Data Layer** consists of centralized cloud-based databases (e.g., Firebase or MySQL) that store and manage entities such as student profiles, driver records, bus and route data, attendance logs, and late form submissions. This layered architecture ensures modularity, scalability, and real-time data synchronization across all user roles, supporting efficient and responsive transportation operations.

CHAPTER 5

DEVELOPMENT TOOLS

5.1 GENERAL

This chapter is about the software language and the tools used in the development of the project. The platform used here is JAVA. The Primary languages are JAVA, J2EE and J2ME. In this project J2EE is chosen for implementation.

5.2 FEATURES OF JAVA

5.2.1 THE JAVA FRAMEWORK

Java is a programming language originally developed by James Gosling at Sun Microsystems and released in 1995 as a core component of Sun Microsystems' Java platform. The language derives much of its syntax from C and C++ but has a simpler object model and fewer low-level facilities. Java applications are typically compiled to byte code that can run on any Java Virtual Machine (JVM) regardless of computer architecture. Java is general-purpose, concurrent, class-based, and object-oriented, and is specifically designed to have as few implementation dependencies as possible. It is intended to let application developers "write once, run anywhere". Java is considered by many as one of the most influential programming languages of the 20th century, and is widely used from application software to web applications the java framework is a new platform independent that simplifies application development internet. Java technology's versatility, efficiency, platform portability, and security make it the ideal technology for network computing. From laptops to datacentres, game consoles to scientific supercomputers, cell phones to the Internet, Java is everywhere!

5.2.2 OBJECTIVES OF JAVA

To see places of Java in Action in our daily life, explore java.com.

WHY SOFTWARE DEVELOPERS CHOOSE JAVA

Java has been tested, refined, extended, and proven by a dedicated community. And numbering more than 6.5 million developers, it's the largest and most active on the planet. With its versatility, efficiency, and portability, Java has become invaluable to developers by enabling them to:

- Write software on one platform and run it on virtually any other platform.
- Create programs to run within a Web browser and Web services.
- Develop server-side applications for online forums, stores, polls, HTML forms processing, and more.
- Combine applications or services using the Java language to create highly customized applications or services.
- Write powerful and efficient applications for mobile phones, remote processors, low-cost consumer products, and practically any other device with a digital heartbeat.

Some Ways Software Developers Learn Java

Today, many colleges and universities offer courses in programming for the Java platform. In addition, developers can also enhance their Java programming skills by reading Sun's java.sun.com Web site, subscribing to Java technology-focused newsletters, using the Java Tutorial and the New to Java Programming Centre, and signing up for Web, virtual, or instructor-led courses.

Object Oriented

To be an Object-Oriented language, any language must follow at least the four characteristics.

1. Inheritance: It is the process of creating the new classes and using the behaviour of the existing classes by extending them just to reuse the existing code and adding addition a feature as needed.

2. Encapsulation: It is the mechanism of combining the information and providing the abstraction.

3. Polymorphism: As the name suggest one name multiple form, Polymorphism is the way of providing the different functionality by the functions having the same name based on the signatures of the methods.

4. Dynamic binding: Sometimes we don't have the knowledge of objects about their specific types while writing our code. It is the way of providing the maximum functionality to a program about the specific type at runtime.

5.2.3 JAVA SERVER PAGES – An Overview

Java Server Pages or JSP for short is Sun's solution for developing dynamic web sites. JSP provide excellent server-side scripting support for creating database driven web applications. JSP enable the developers to directly insert java code into JSP file, this makes the development process very simple, and its maintenance also becomes very easy. JSP pages are efficient, it loads into the web server's memory on receiving the request very first time and the subsequent calls are served within a very short period. In today's environment most web sites servers' dynamic pages based on user request. Database is very convenient way to store the data of users and other things. JDBC provide excellent database connectivity in heterogeneous database environment. Using JSP and JDBC its very cc easy to develop database driven web application. Java is known for its characteristic of "write once, run anywhere." JSP pages are platform Java Server Pages.Java Server Pages (JSP) technology is the Java platform technology for delivering dynamic content to web clients in a portable, secure and well-defined way. The Java Server Pages specification extends the Java Servlet API to provide web application developers with a robust framework for creating dynamic web content on the server using HTML, and XML templates, and Java code, which is secure, fast, and independent of server platforms. JSP has been built on top of the Servlet API and utilizes Servlet semantics. JSP has become the preferred request handler and response mechanism. Although JSP technology is going to be a powerful successor to basic Servlets, they have an evolutionary relationship and can be used in a cooperative and complementary manner. Servlets are powerful and sometimes they are a bit cumbersome when it comes to generating complex HTML. Most servlets contain a little code that handles application logic and a lot more code that handles output formatting. This can make it difficult to separate and reuse portions of the code when a different output format is needed. For these reasons, web application developers turn towards JSP as their preferred servlet environment.

5.2.4 EVOLUTION OF ANDROID APPLICATIONS

Over the last few years, web server applications have evolved from static to dynamic applications. This evolution became necessary due to some deficiencies in earlier web site design. For example, to put more of business processes on the web, whether in business-to-consumer (B2C) or business-to-business (B2B) markets, conventional web site design technologies are not enough. The main issues, every developer faces when developing web applications, are:

1. Scalability - a successful site will have more users and as the number of users is increasing Fastly, the web applications must scale correspondingly.

2. Integration of data and business logic - the web is just another way to conduct business, and so it should be able to use the same middle-tier and data-access code.

3. Manageability - web sites just keep getting bigger and we need some viable mechanism to manage the ever-increasing content and its interaction with business systems.

4. Personalization - adding a personal touch to the web page becomes an essential factor to keep our customer coming back again. Knowing their preferences, allowing them to configure the information they view, remembering their past transactions or frequent search keywords are all important in providing feedback and interaction from what is otherwise a fairly one-sided conversation.

Apart from these general needs for a business-oriented web site, the necessity for new technologies to create robust, dynamic and compact server-side web applications has been realized. The main characteristics of today's dynamic web server applications are as follows:

1. Serve HTML and XML, and stream data to the web client.

2. Separate presentation, logic and data.

3. Interface to databases, other Java applications, CORBA, directory and mail services.

4. Make use of application server middleware to provide transactional support.

5. Track client sessions.

5.2.5 HISTROY OF ANDROID

The history and versions of android are interesting to know. The code names of android ranges from A to J currently , such as **Aestro**, **Blender**, **Cupcake**, **Donut**, **Eclair**, **Froyo**, **Gingerbread**, **Honeycomb**, **IceCreamSandwitch**, **Jellybean**, **KitKat** and **Lollipop**. Let's understand android history in a sequence.

1) Initially, **Andy Rubin** founded Android Incorporation in Palo Alto, California, United States in October 2003.

2) On 17th August 2005, Google acquired android Incorporation. Since then, it is in the subsidiary of Google Incorporation.

3) The key employees of Android Incorporation are **Andy Rubin**, **Rich Miner**, **Chris White** and **Nick Sears**.

5.2.6 ANDROID VERSIONS, CODENAME AND API

Version	Code name	API Level
1.5	Cupcake	3
1.6	Donut	4
2.1	Eclair	7
2.2	Froyo	8

2.3	Gingerbread	9 and 10
3.1 and 3.3	Honeycomb	12 and 13
4.0	Ice Cream Sandwitch	15
4.1, 4.2 and 4.3	Jelly Bean	16, 17 and 18
4.4	KitKat	19
5.0	Lollipop	21
6.0	Marshmallow	23
7.0	Nougat	24-25
8.0	Oreo	26-27

5.2.7 BENEFITS OF JAVA

One of the main reasons why the Java Server Pages technology has evolved into what it is today, and it is still evolving is the overwhelming technical need to simplify application design by separating dynamic content from static template display data. Another benefit of utilizing JSP is that it allows to more cleanly separate the roles of web application/HTML designer from a software developer. The JSP technology is blessed with a few exciting benefits, which are chronicled as follows:

1. The JSP technology is platform independent, in its dynamic web pages, its web servers, and its underlying server components. That is, JSP pages perform perfectly without any hassle on any platform, run on any web server, and web-enabled application server. The JSP pages can be accessed from any web server.

2. The JSP technology emphasizes the use of reusable components. These components can be combined or manipulated towards developing more purposeful components and page design. This definitely reduces development time apart from the At development time, JSPs are very different from Servlets, however, they are precompiled into Servlets at run time and executed by a JSP engine which is installed on a Web-enabled application server such as BEA WebLogic and IBM WebSphere.

5.3 LAYOUT

Earlier in client- server computing, each application had its own client program, and it worked as a user interface and need to be installed on each user's personal computer. Most web applications use HTML/XHTML that are mostly supported by all the browsers and web pages are displayed to the client as static documents. A web page can merely display static content and it also lets the user navigate through the content, but a web application provides a more interactive experience. Any computer running Servlets or JSP needs to have a container. A container is nothing but a piece of software responsible for loading, executing and unloading the Servlets and JSP. While servlets can be used to extend the functionality of any Java- enabled server. They are mostly used to extend web servers and are efficient replacement for CGI scripts. CGI was one of the earliest and most prominent server-side dynamic content solutions, so before going forward it is very important to know the difference between CGI and the Servlets.

5.4 ANDROID XML

XML stands for Extensible Markup Language. XML is a markup language much like HTML used to describe data. It is derived from Standard Generalized Markup Language (SGML). Basically, the XML tags are not predefined in XML. We need to implement and define the tags in XML. XML tags define the data and use to store and organize data. It’s easily scalable and simple to develop. In Android, the XML is used to implement UI-related data, and it’s a lightweight markup language that doesn’t make layout heavy. XML only contains tags, while implementing they need to be just invoked.

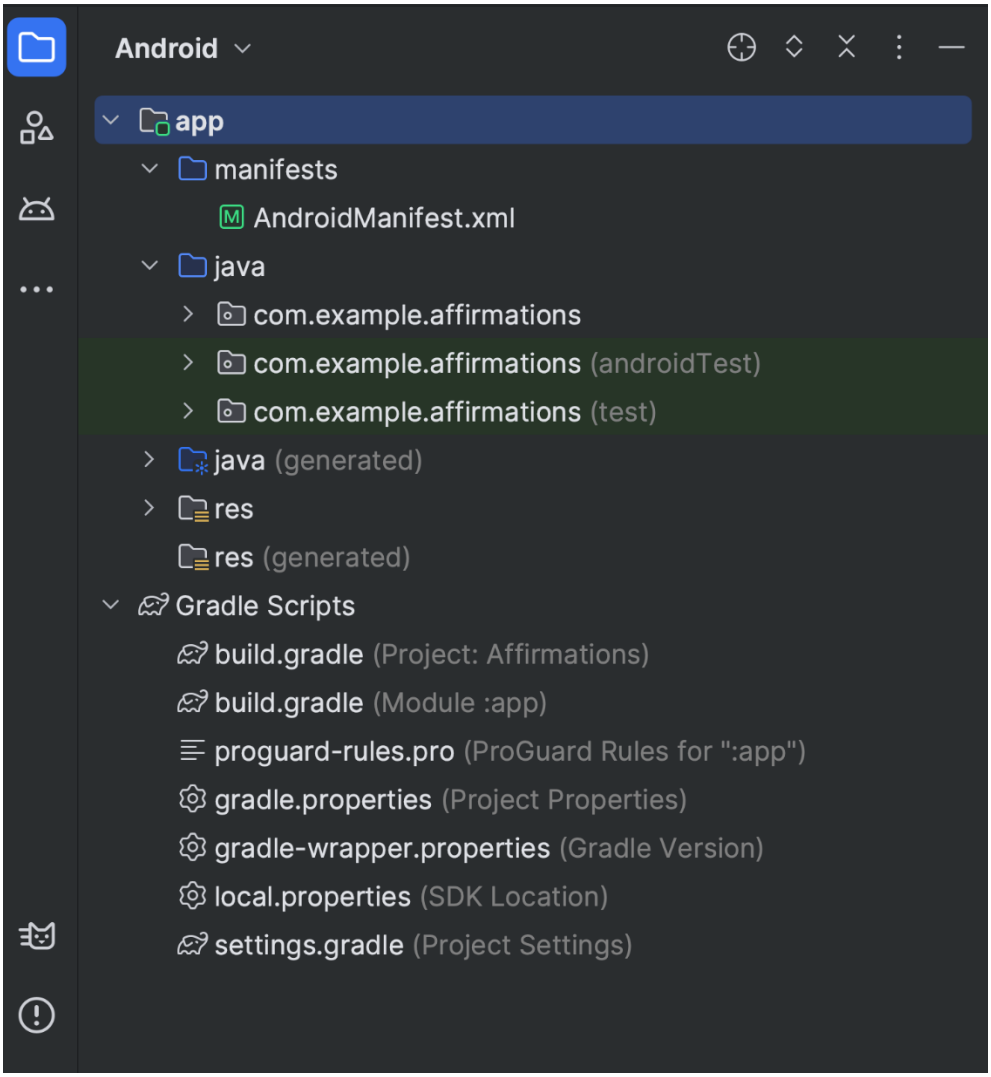


Fig.13. Project files in Android project view.

Each project in Android Studio contains one or more modules with source code files and resource files. The types of modules include:

- Android app modules
- Library modules
- Google App Engine modules

By default, Android Studio displays your project files in the Android project view, as shown in figure 1. This view is organized by modules to provide quick access to your project's key source files. All the build files are visible at the top level, under **Gradle Scripts**.

Each app module contains the following folders

- **manifests**: Contains the AndroidManifest.xml file.

- **java**: Contains the Kotlin and Java source code files, including JUnit test code.
- **res**: Contains all non-code resources such as UI strings and bitmap images.

The Android project structure on disk differs from this flattened representation. To see the actual file structure of the project, select **Project** instead of **Android** from the **Project** menu.

For more information, see Projects overview.

Gradle build system

Android Studio uses Gradle as the foundation of the build system, with more Android-specific capabilities provided by the Android Gradle plugin. This build system runs as an integrated tool from the Android Studio menu and independently from the command line. You can use the features of the build system to do the following:

- Customize, configure, and extend the build process.
- Create multiple APKs for your app with different features, using the same project and modules.
- Reuse code and resources across source sets.

By employing the flexibility of Gradle, you can achieve all of this without modifying your app's core source files.

Android Studio build files are named `build.gradle.kts` if you use Kotlin (recommended) or `build.gradle` if you use Groovy. They are plain text files that use the Kotlin or Groovy syntax to configure the build with elements provided by the Android Gradle plugin. Each project has one top-level build file for the entire project and separate module-level build files for each module. When you import an existing project, Android Studio automatically generates the necessary build files.

Note: We might reference either the **build.gradle.kts** or **build.gradle** file alone in the documentation, but they're conceptually interchangeable. For example if you see **build.gradle.kts** but you use the Groovy DSL to configure your build, you can think of it as the **build.gradle** file (and the other way around).

To learn more about the build system and how to configure your build, see Configure your build.

Build variants

The build system can help you create different versions of the same app from a single project. This is useful when you have both a free version and a paid version of your app or if you want to distribute multiple APKs for different device configurations on Google Play.For more information about configuring build variants, see Configure build variants.

5.5 CONCLUSION

XML and layout are gaining rapid acceptance as means to provide dynamic content on the Internet. With full access to the Java platform, running from the server in a secure manner, the application possibilities are almost limitless. When JSPs are used with Enterprise JavaBeans technology, e-commerce and database resources can be further enhanced to meet an enterprise's needs for web applications providing secure transactions in an open platform. J2EE technology as a whole makes it easy to develop, deploy and use web server applications instead of mingling with other technologies such as CGI and ASP. There are many tools for facilitating quick web software development and to easily convert existing server-side technologies to JSP and Servlets.

CHAPTER 6

IMPLEMENTATION

3.1 GENERAL

This chapter describes the implementation of searched based application. It deals with the source code for main viewpoint for Anonymous Database Management.

3.2 CODINGS

```
IMPORT 'PACKAGE:COLLEGEBUSTRACKING/UTILLS/AUTH_GATE.DART';
IMPORT 'PACKAGE:COLLEGEBUSTRACKING/UTILLS/AUTH_GATE2.DART';
IMPORT 'PACKAGE:COLLEGEBUSTRACKING/UTILLS/AUTH_GATE3.DART';
IMPORT 'PACKAGE:FIREBASE_CORE/FIREBASE_CORE.DART';
IMPORT 'PACKAGE:FLUTTER/MATERIAL.DART';
FUTURE<VOID> MAIN() ASYNC {
 WIDGETSFLUTTERBINDING.ENSUREINITIALIZED();
 AWAIT FIREBASE.INITIALIZEAPP();
 RUNAPP(MYAPP());
}

CLASS MYAPP EXTENDS STATELESSWIDGET {
 @OVERRIDE
 WIDGET BUILD(BUILDCONTEXT CONTEXT) {
  RETURN MATERIALAPP(
   ROUTES: {
    '/FIRST': (CONTEXT) => MYAPP(),
    '/SECOND': (CONTEXT) => HOMESCREEN(),
   }
   HOME: SPLASHSCREEN(),
  );
 }
```

```
}
CLASS SPLASHSCREEN EXTENDS STATELESSWIDGET {
  FUTURE<VOID> SIMULATEASYNCOPERATION() ASYNC {
    AWAIT FUTURE.DELAYED(DURATION(SECONDS: 5));
  }

  @OVERRIDE
  WIDGET BUILD(BUILDCONTEXT CONTEXT) {
    RETURN SCAFFOLD(
      BODY: STACK(
        FIT: STACKFIT.EXPAND,
        CHILDREN: [
          // BACKGROUND IMAGE
          IMAGE.ASSET(
            'ASSETS/IMAGES/T3.JPG', // REPLACE WITH YOUR IMAGE PATH
            FIT: BOXFIT.COVER,
          ),
          // SPLASH SCREEN OVERLAY
          CONTAINER(
            COLOR: COLORS.BLACK.WITHOPACITY(0.7), // ADJUST OPACITY AS NEEDED
          ),
          CENTER(
            CHILD: FUTUREBUILDER(
              FUTURE: SIMULATEASYNCOPERATION(),
              BUILDER: (CONTEXT, SNAPSHOT) {
                IF (SNAPSHOT.CONNECTIONSTATE == CONNECTIONSTATE.DONE) {
                  RETURN HOMESCREEN();
                } ELSE {
                  RETURN CENTER(
                    CHILD: COLUMN(
                      MAINAXISALIGNMENT: MAINAXISALIGNMENT.CENTER,
                      CHILDREN: [
                        TEXT(
```

```
                  'WELCOME, LETS SAFE DRIVE',
                  STYLE: TEXTSTYLE(
                    FONTWEIGHT: FONTWEIGHT.BOLD,
                    FONTSIZE: 20.0,
                    COLOR: COLORS.YELLOW,
                  ),
                ),
                SIZEDBOX(HEIGHT: 20.0),
                CIRCULARPROGRESSINDICATOR(
                  COLOR: COLORS.WHITE,
                ),
              ],
            ),
          );
        }
      },
    ),
  ),
 ],
 ),
 );
 }
}

CLASS HOMESCREEN EXTENDS STATELESSWIDGET {
 @OVERRIDE
 WIDGET BUILD(BUILDCONTEXT CONTEXT) {
  RETURN SCAFFOLD(
   BACKGROUNDCOLOR: COLORS.TRANSPARENT,
   BODY: CENTER(
    CHILD: COLUMN(
     MAINAXISALIGNMENT: MAINAXISALIGNMENT.CENTER,
     CHILDREN: [
```

```
          FADEIN(
            CHILD: TEXT(
              '',
              STYLE: TEXTSTYLE(
                FONTSIZE: 24.0,
                FONTWEIGHT: FONTWEIGHT.BOLD,
                COLOR: COLORS.WHITE,
              ),
            ),
          ),
          SIZEDBOX(HEIGHT: 40.0),
          STAGGEREDANIMATIONBUTTONS(),
        ],
      ),
    ),
  );
 }
}

CLASS FADEIN EXTENDS STATEFULWIDGET {
 FINAL WIDGET CHILD;
 FADEIN({REQUIRED THIS.CHILD});

 @OVERRIDE
 _FADEINSTATE CREATESTATE() => _FADEINSTATE();
}
CLASS _FADEINSTATE EXTENDS STATE<FADEIN> WITH SINGLETICKERPROVIDERSTATEMIXIN {
 LATE ANIMATIONCONTROLLER _ANIMATIONCONTROLLER;
 LATE ANIMATION<DOUBLE> _FADEINANIMATION;

 @OVERRIDE
 VOID INITSTATE() {
  SUPER.INITSTATE();
```

```
    _ANIMATIONCONTROLLER = ANIMATIONCONTROLLER(
      VSYNC: THIS,
      DURATION: DURATION(SECONDS: 1),
    );

    _FADEINANIMATION = TWEEN<DOUBLE>(BEGIN: 0.0, END: 1.0).ANIMATE(
      CURVEDANIMATION(
        PARENT: _ANIMATIONCONTROLLER,
        CURVE: CURVES.EASEINOUT,
      ),
    );

    _ANIMATIONCONTROLLER.FORWARD();
  }
  @OVERRIDE
  WIDGET BUILD(BUILDCONTEXT CONTEXT) {
    RETURN FADETRANSITION(
      OPACITY: _FADEINANIMATION,
      CHILD: WIDGET.CHILD,
    );
  }

  @OVERRIDE
  VOID DISPOSE() {
    _ANIMATIONCONTROLLER.DISPOSE();
    SUPER.DISPOSE();
  }
}

CLASS STAGGEREDANIMATIONBUTTONS EXTENDS STATELESSWIDGET {
  @OVERRIDE
  WIDGET BUILD(BUILDCONTEXT CONTEXT) {
```

```
RETURN COLUMN(
  CHILDREN: [
    SIZEDBOX(HEIGHT: 20,),



    STAGGEREDBUTTON(
      LABEL: 'DRIVER',
      ICONDATA: ICONS.DRIVE_ETA,
      ONPRESSED: () {
        // HANDLE BUTTON 2 ACTION
        NAVIGATOR.PUSHREPLACEMENT(
          CONTEXT,
          MATERIALPAGEROUTE(BUILDER: (CONTEXT) => AUTHGATE2()),
        );
      },
    ),

    SIZEDBOX(HEIGHT: 20,),
    STAGGEREDBUTTON(
      LABEL: 'STUDENT',
      ICONDATA: ICONS.PERSON_ADD,
      ONPRESSED: () {
        // HANDLE BUTTON 3 ACTION
        NAVIGATOR.PUSHREPLACEMENT(
          CONTEXT,
          MATERIALPAGEROUTE(BUILDER: (CONTEXT) => AUTHGATE()),
        );
      },
    ),

    SIZEDBOX(HEIGHT: 20,),
    // STAGGEREDBUTTON(
    //   LABEL: 'ADMIN',
```

```
      //   ICONDATA: ICONS.ADMIN_PANEL_SETTINGS_ROUNDED,
      //   ONPRESSED: () {
      //     // HANDLE BUTTON 3 ACTION
      //     // NAVIGATOR.PUSHREPLACEMENT(
      //     //   CONTEXT,
      //     //   MATERIALPAGEROUTE(BUILDER: (CONTEXT) => AUTHGATE3()),
      //     // );
      //   },
      // ),
    ],
   );
  }
}

CLASS STAGGEREDBUTTON EXTENDS STATEFULWIDGET {
  FINAL STRING LABEL;
  FINAL VOIDCALLBACK ONPRESSED;
  FINAL ICONDATA ICONDATA;

  STAGGEREDBUTTON({
    REQUIRED THIS.LABEL,
    REQUIRED THIS.ONPRESSED,
    REQUIRED THIS.ICONDATA,
  });

  @OVERRIDE
  _STAGGEREDBUTTONSTATE CREATESTATE() => _STAGGEREDBUTTONSTATE();
}

CLASS _STAGGEREDBUTTONSTATE EXTENDS STATE<STAGGEREDBUTTON>
    WITH SINGLETICKERPROVIDERSTATEMIXIN {
  LATE ANIMATIONCONTROLLER _ANIMATIONCONTROLLER;
  LATE ANIMATION<DOUBLE> _SCALEANIMATION;
```

```
@OVERRIDE
VOID INITSTATE() {
  SUPER.INITSTATE();
  _ANIMATIONCONTROLLER = ANIMATIONCONTROLLER(
    VSYNC: THIS,
    DURATION: DURATION(MILLISECONDS: 500),
  );
  _SCALEANIMATION = TWEEN<DOUBLE>(BEGIN: 0.0, END: 1.0).ANIMATE(
    CURVEDANIMATION(
      PARENT: _ANIMATIONCONTROLLER,
      CURVE: CURVES.EASEINOUTBACK,
    ),
  );
  _ANIMATIONCONTROLLER.FORWARD();
}

@OVERRIDE
WIDGET BUILD(BUILDCONTEXT CONTEXT) {
  RETURN SCALETRANSITION(
    SCALE: _SCALEANIMATION,
    CHILD: ELEVATEDBUTTON(
      ONPRESSED: WIDGET.ONPRESSED,
      STYLE: ELEVATEDBUTTON.STYLEFROM(
        PRIMARY: COLORS.BLACK,
        ONPRIMARY: COLORS.INDIGO.SHADE900,
        PADDING: EDGEINSETS.ALL(10),
        SHAPE: ROUNDEDRECTANGLEBORDER(
          BORDERRADIUS: BORDERRADIUS.CIRCULAR(10.0),
        ),
      ),
      CHILD: ROW(
        MAINAXISALIGNMENT: MAINAXISALIGNMENT.CENTER,
```

```
        CHILDREN: [
          ICON(
            WIDGET.ICONDATA,
            COLOR: COLORS.YELLOW,
            SIZE: 34,
          ),
          SIZEDBOX(WIDTH: 20), // ADJUST THE SPACING BETWEEN ICON AND LABEL
          TEXT(
            WIDGET.LABEL,
            STYLE: TEXTSTYLE(
              COLOR: COLORS.GREY, // SET THE TEXT COLOR
              FONTSIZE: 34.0,
              FONTWEIGHT: FONTWEIGHT.BOLD,
            ),
          ),
        ],
      ),
    ),
  );
 }

 @OVERRIDE
 VOID DISPOSE() {
  _ANIMATIONCONTROLLER.DISPOSE();
  SUPER.DISPOSE();
 }
}
IMPORT 'DART:ASYNC';
IMPORT 'DART:TYPED_DATA';
IMPORT 'PACKAGE:FIREBASE_AUTH/FIREBASE_AUTH.DART';
IMPORT 'PACKAGE:FIREBASE_DATABASE/FIREBASE_DATABASE.DART';
IMPORT 'PACKAGE:FLUTTER/CUPERTINO.DART';
IMPORT 'PACKAGE:FLUTTER/MATERIAL.DART';
```

```
IMPORT 'PACKAGE:FLUTTER/SERVICES.DART';
IMPORT 'PACKAGE:FLUTTER_IMAGE_COMPRESS/FLUTTER_IMAGE_COMPRESS.DART';
IMPORT 'PACKAGE:GOOGLE_MAPS_FLUTTER/GOOGLE_MAPS_FLUTTER.DART';
IMPORT 'PACKAGE:URL_LAUNCHER/URL_LAUNCHER.DART';

CLASS FULLVIEWPAGE EXTENDS STATEFULWIDGET {
  FINAL STRING AKEY;
  FULLVIEWPAGE(
{
    REQUIRED THIS.AKEY,
  });


  @OVERRIDE
  _FULLLISTPAGESTATE CREATESTATE() => _FULLLISTPAGESTATE();
}

CLASS _FULLLISTPAGESTATE EXTENDS STATE<FULLVIEWPAGE> {
  STRING SELECTEDRADIO = ''; // VARIABLE TO HOLD THE SELECTED RADIO VALUE
  FINAL FIREBASEAUTH _AUTH = FIREBASEAUTH.INSTANCE;
  FINAL DATABASEREFERENCE _DATABASE = FIREBASEDATABASE.INSTANCE.REFERENCE();
  LATE GOOGLEMAPCONTROLLER MAPCONTROLLER;
  DATABASEREFERENCE REFERENCE = FIREBASEDATABASE.INSTANCE.REFERENCE();
  STREAMCONTROLLER<MAP<STRING, DYNAMIC>> _LOCATIONSTREAMCONTROLLER =
  STREAMCONTROLLER<MAP<STRING, DYNAMIC>>();
  UINT8LIST? COMPRESSEDBYTES; // DECLARE COMPRESSEDBYTES HERE

  @OVERRIDE
  VOID INITSTATE() {
    SUPER.INITSTATE();
    STARTLOCATIONUPDATES();
    LOADCOMPRESSEDBYTES(); // LOAD COMPRESSED BYTES WHEN THE WIDGET INITIALIZES
  }
```

```
@OVERRIDE
VOID DISPOSE() {
  _LOCATIONSTREAMCONTROLLER.CLOSE();
  SUPER.DISPOSE();
}

FUTURE<VOID> LOADCOMPRESSEDBYTES() ASYNC {
  // LOAD THE IMAGE BYTE DATA
  BYTEDATA DATA = AWAIT ROOTBUNDLE.LOAD('ASSETS/IMAGES/AMBULANCE.PNG');
  // CONVERT LIST<INT> TO UINT8LIST
  UINT8LIST ORIGINALBYTES = UINT8LIST.FROMLIST(DATA.BUFFER.ASUINT8LIST());
  // COMPRESS THE IMAGE TO REDUCE SIZE
  COMPRESSEDBYTES = AWAIT FLUTTERIMAGECOMPRESS.COMPRESSWITHLIST(
    ORIGINALBYTES,
    MINHEIGHT: 68, // SET YOUR DESIRED HEIGHT
    MINWIDTH: 68, // SET YOUR DESIRED WIDTH
    QUALITY: 80, // SET YOUR DESIRED QUALITY (0 TO 100)
  );
}

VOID STARTLOCATIONUPDATES() {
  DATABASEREFERENCE REFERENCE =
  FIREBASEDATABASE.INSTANCE.REFERENCE().CHILD('LOCATIONS').CHILD(WIDGET.AKEY);

  REFERENCE.ONVALUE.LISTEN((EVENT) {
    IF (EVENT.SNAPSHOT.VALUE != NULL) {
      VAR USERDATA = EVENT.SNAPSHOT.VALUE;
      // USE NULL-AWARE CAST TO MAP<STRING, DYNAMIC>
      IF (USERDATA IS MAP<OBJECT?, OBJECT?>) {
        // CAST THE MAP TO MAP<STRING, DYNAMIC>
        MAP<STRING, DYNAMIC> USERDATAMAP = USERDATA.CAST<STRING, DYNAMIC>();
        // ADD THE DATA TO THE STREAM
```

```
      _LOCATIONSTREAMCONTROLLER.ADD(USERDATAMAP);

      // SHOW LOCATION ON THE MAP CONTINUOUSLY
      SHOWLOCATIONONMAP(USERDATAMAP['LATITUDE'], USERDATAMAP['LONGITUDE']);
     } ELSE {
      PRINT("INVALID DATA STRUCTURE IN THE DATABASE");
     }
    } ELSE {
     PRINT("NO LOCATION DATA FOUND IN THE DATABASE");
    }
   }, ONERROR: (OBJECT ERROR) {
    PRINT("ERROR FETCHING LOCATION DATA: $ERROR");
   });
}
@OVERRIDE
WIDGET BUILD(BUILDCONTEXT CONTEXT) {
 RETURN SCAFFOLD(
   APPBAR: APPBAR(
    TITLE: TEXT('DETAILS'),
   ),
   BODY:
            STREAMBUILDER<MAP<STRING, DYNAMIC>>(
              STREAM: _LOCATIONSTREAMCONTROLLER.STREAM,
              BUILDER: (CONTEXT, SNAPSHOT) {
               IF (SNAPSHOT.HASDATA) {
                DOUBLE? LATITUDE = SNAPSHOT.DATA!['LATITUDE'];
                DOUBLE? LONGITUDE = SNAPSHOT.DATA!['LONGITUDE'];
                RETURN CONTAINER(
                 HEIGHT: DOUBLE.INFINITY,
                 WIDTH: DOUBLE.INFINITY,
                 CHILD: GOOGLEMAP(
                  INITIALCAMERAPOSITION: CAMERAPOSITION(
                   TARGET: LATLNG(LATITUDE!, LONGITUDE!),
```

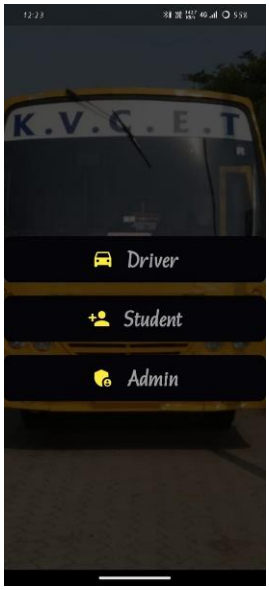
```
                ZOOM: 14.0,
              ),
              ONMAPCREATED: (GOOGLEMAPCONTROLLER CONTROLLER) {
                MAPCONTROLLER = CONTROLLER;
              },
              MARKERS: {
                MARKER(
                  MARKERID: MARKERID('LOCATIONMARKER'),
                  POSITION: LATLNG(LATITUDE, LONGITUDE),
                  ICON: BITMAPDESCRIPTOR.FROMBYTES(COMPRESSEDBYTES!),
                  INFOWINDOW: INFOWINDOW(TITLE: 'PATIENT LOCATION'),
                ),
              },
            ),
          );
        } ELSE {
          // SHOW LOADING OR DEFAULT MAP
          RETURN CONTAINER();
        }
      },
    ),
  );
}
```

CHAPTER 7

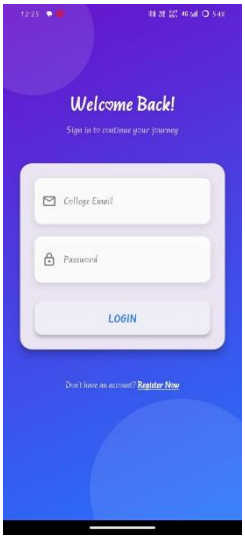
7.1 GENERAL

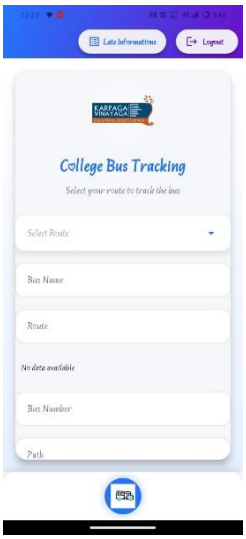
In this Flutter-Based Transportation Management System, snapshots are visual representations of key features and interfaces used by students, drivers, and admins. They help illustrate the app's functionality, user interactions, and overall design, serving as a visual record of the system's development and usability.

7.2 VARIOUS SNAPSHOTS

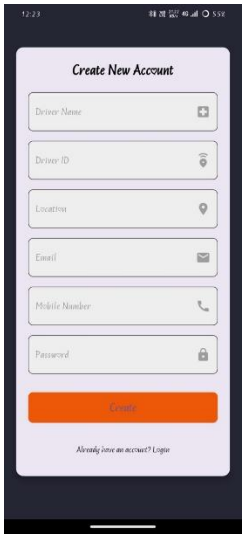


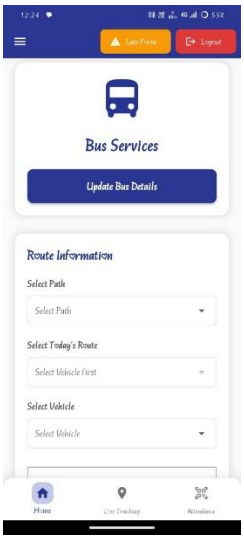
STUDENT MODULE

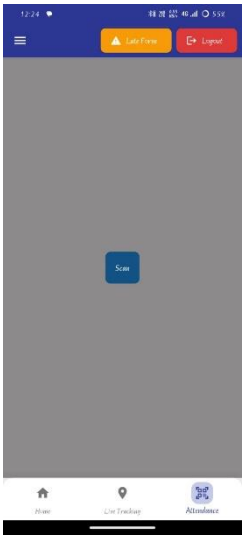




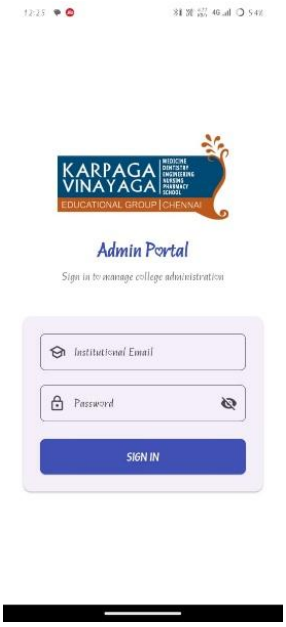
DRIVER MODULE







ADMIN MODULE



CHAPTER 8

SOFTWARE TESTING

8.1. FIREBASE

Firebase is a Backend-as-a-Service (Baas). It provides developers with a variety of tools and services to help them develop quality apps, grow their user base, and earn profit. It is built on Google's infrastructure.Firebase is categorized as a NoSQL database program, which stores data in JSON-like documents.

In Firebase, a document is a set of key-value pairs defined by a schema. A group of documents makes up a collection.

Key Features

1. Authentication

It supports authentication using passwords, phone numbers, Google, Facebook, Twitter, and more. Firebase Authentication (SDK) can be used to manually integrate one or more sign-in methods into an app.

2. Realtime database

Data is synced across all clients in realtime and remains available even when an app goes offline.

3. Hosting

Firebase Hosting provides fast hosting for a web app; content is cached into content delivery networks worldwide.

4. Test lab

The application is tested on virtual and physical devices located in Google's data centers.

5. Notifications

Notifications can be sent with firebase with no additional coding.Users can get started with firebase for free; more details can be found on the official website.

8.2. FEASIBILITY STUDY

Feasibility studies aim to objectively and rationally uncover the strengths and weaknesses of the existing business or proposed venture, opportunities and threats as presented by the environment, the resources required to carry through, and ultimately the prospects for success. In its simplest term, the two criteria to judge feasibility are cost required and value to be attained. As such, a well-designed feasibility study should provide a historical background of the business or project, description of the product or service, accounting statements, details of the operations and management, marketing research and policies, financial data, legal requirements and tax obligations. Generally, feasibility studies precede technical development and project implementation.

There are 3 types of Feasibility

- Economical feasibility
- Technical feasibility
- Operational feasibility

8.2.1. ECONOMICAL FEASIBILITY

The assessment is based on an outline design of system requirements in terms of Input, Processes, Output, Fields, Programs, and Procedures. This can be quantified in terms of volumes of data, trends, frequency of updating, etc. in order to estimate whether the new system will perform adequately or not.

8.2.2. TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources.

8.2.3 OPERATIONAL FEASIBILITY:

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity.

8.3. SYSTEM TESTING

The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design. In the real sense it is the phase which helps to bring out the creativity of the other phases that makes it shine.

8.3.1. VARIOUS LEVELS OF TESTING

1. White Box Testing
2. Black Box Testing
3. Unit Testing
4. Functional Testing
5. Performance Testing
6. Integration Testing
7. Validation Testing
8. System Testing
9. Output Testing
10. User Acceptance Testing

8.3.1.1. WHITE BOX TESTING

White-box testing, sometimes called glass-box, is a test case design method that uses the control structure of the procedural design to derive test cases. Using White Box testing methods, we can derive test cases that

• Guarantee that all independent paths within a module have been exercised at least once

• Exercise all logical decisions on their true and false sides.

• Execute all loops at their boundaries and within their operational bounds.

• Exercise internal data structures to assure their validity.

8.3.1.2. BLACK BOX TESTING

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, like most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a test in which the software under the test is treated as a black box. You cannot “see” into it. The test provides input and responds to outputs without considering how the software works. In this testing by knowing the internal operation of a product, test can be conducted to ensure that “all gears mesh”, that is the internal operation performs according to specification and all internal components have been adequately exercised. It fundamentally focuses on the functional requirements of the software.

8.3.1.3. UNIT TESTING

Unit testing is a method by which individual units of source code, sets of one or more computer program modules together with associated control data, usage procedures, and operating procedures are tested to determine if they are fit for use. Intuitively, one can view a unit as the smallest testable part of an application. In procedural programming, a unit could be an entire module, but it is more commonly an individual function or procedure. In object-oriented programming, a unit is often an entire interface, such as a class, but could be an individual method. Unit tests are short code fragments created by programmers or occasionally by white box testers during the development process. Unit testing is software verification and validation method in which the individual units of source code are tested fit for use. A unit is the smallest testable part of an

application. In this testing, each class is tested to be working satisfactorily. Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. it is done after the completion of an individual unit before integration.

8.3.1.4. FUNCTIONAL TESTING:

Functional testing is a quality assurance (QA) process and a type of black box testing that bases its test cases on the specifications of the software component under test. Functions are tested by feeding them input and examining the output, and internal program structure is rarely considered (not like in white-box testing). Functional Testing usually describes what the system does. Functional testing differs from system testing in that functional testing "verifies a program by checking it against ... design document(s) or specification(s)", while system testing "validate a program by checking it against the published user or system requirements" (Kane, Falk, Nguyen 1999, p. 52). Functional testing typically involves five steps. The identification of functions that the software is expected to perform

1. The creation of input data based on the function's specifications

2. The determination of output based on the function's specifications

3. The execution of the test case

4. The comparison of actual and expected outputs.

8.3.1.5. PERFORMANCE TESTING

In general testing is performed to determine how a system performs in terms of responsiveness and stability under a particular workload. It can also serve to investigate, measure, validate or verify other quality attributes of the system, such as scalability, reliability and resource usage.

Performance testing is a subset of performance engineering, an emerging computer science practice which strives to build performance into the implementation, design and architecture of a system.

8.3.1.6. INTEGRATION TESTING

Integration testing is a systematic technique for constructing the program structure while at the same time conducting tests to uncover errors associated with. Individual modules, which are highly prone to interface errors, should not be assumed to work instantly when put together. The problem of course, is “putting them together”- interfacing. There may be the chances of data loss across on another’s sub functions, when combined may not produce the desired major function; individually acceptable impression may be magnified to unacceptable levels; global data structures can present problems. Integration testing is the phase in software testing in which individual software modules are combined and tested as a group. Integration testing takes as its input modules that have been unit tested, groups them in larger aggregates, applies tests defined in an integration test plan to those aggregates, and delivers as its output the integrated system ready. All the errors found in the system are corrected for the next phase. The purpose of integration testing is to verify functional, performance, and reliability requirements placed on major design items. These "design items", i.e. assemblages (or groups of units), are exercised through their interfaces using black box testing, success and error cases being simulated via appropriate parameter and data inputs. Simulated usage of shared data areas and inter-process communication is tested, and individual subsystems are exercised through their input interface. Test cases are constructed to test whether all the components within assemblages interact correctly, for example across procedure calls or process activations, and this is done after testing individual modules, i.e. unit testing.

8.3.1.7. VALIDATION TESTING

Verification and Validation are independent procedures that are used together for checking that a product, service, or system meets requirements and specifications and that it full fills its intended purpose. These are critical components of a quality management system such as ISO 9000. The words "verification" and "validation" are sometimes preceded with "Independent" (or IV&V), indicating that the verification and validation is to be performed by a disinterested third party. It is sometimes said that validation can be expressed by the query "Are you building the right thing?" and verification by "Are you building it right?". In practice, the usage of these terms varies. Sometimes they are even used interchangeably.

8.3.1.8. SYSTEM TESTING

System testing of software or hardware is testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements. System testing falls within the scope of black box testing, and as such, should require no knowledge of the inner design of the code or logic. As a rule, system testing takes, as its input, all the "integrated" software components that have passed integration testing and the software system itself integrated with any applicable hardware system(s). The purpose of integration testing is to detect any inconsistencies between the software units that are integrated together (called *assemblages*) or between any of the *assemblages* and the hardware. System testing is a more limited type of testing; it seeks to detect defects both within the "inter-assembles" and within the system as a whole.
System testing is performed on the entire system in the context of a Functional Requirement Specification(s) (FRS) and/or a System Requirement Specification (SRS). System testing tests not only the design, but also the behaviour and even the believed expectations of the customer. It is also intended to test up to and beyond the bounds defined in the software/hardware requirements specification.

8.3.1.9. OUTPUT TESTING

After performing the validation testing, the next step is output testing of the proposed system since no system could be useful if it does not produce the required output generated or considered into two ways. One is on screen and another is printed format. The output comes as the specified requirements by the user. Hence output testing does not result in any correction in the system.

8.3.1.10. USER ACCEPTANCE TESTING

User acceptance of a system is the factor for the success of any system. The system under consideration is tested for the user acceptance by constantly keeping in touch with the prospective system users at the time of developing and making changes wherever required.

- Input screen design.
- Output screen design.
- Online message to guide users.

CHAPTER 9

APPLICATION

9.1 GENERAL

A Flutter-based TMS offers a cross-platform solution that ensures seamless communication and coordination among students, drivers, and administrators. Leveraging Flutter's capabilities, such systems provide real-time updates, efficient route management, and enhanced user experiences across Android, iOS, and web platforms.

9.2 APPLICATIONS

The Flutter-based Transportation Management System (TMS) is designed to enhance student transportation services through three specialized modules:

- **Student Module:** Enables students to register, select bus routes based on their location, track buses in real-time via Google Maps, and receive notifications about delays or route changes.
- **Driver Module**: Allows drivers to log in, update trip details, submit delay reports, and manage student attendance by scanning QR codes on student IDs.
- **Admin Module**: Provides administrators with tools to manage drivers, buses, student records, attendance logs, and delay reports, ensuring efficient oversight of transportation logistics.

By integrating real-time tracking, QR code-based attendance, and a user-friendly interface, this system streamlines operations, improves communication among stakeholders, and enhances the reliability of student transportation services.

CHAPTER 10

CONCLUSION

10.1 CONCLUSION

In conclusion, the transportation management system effectively streamlines the operations for students, drivers, and administrators by offering essential features like real-time location tracking, route selection, attendance management, and late form submission. With the potential for future enhancements such as AI-powered route optimization, mobile app improvements, integrated payment systems, and real-time traffic data, the system can evolve to offer a more efficient, user-friendly, and adaptive solution. These improvements would enhance safety, convenience, and overall operational efficiency, ensuring seamless communication and better user experiences for all parties involved

10.2 FUTURE ENHANCEMENTS

Mobile App Optimization: As mobile devices are central to user interaction, enhancing the mobile app experience with offline capabilities, push notifications, and personalized dashboards could improve accessibility and real-time tracking for both students and drivers.

Real-Time Traffic Data Integration: By integrating real-time traffic data from APIs like Google Traffic or Waze, the system could dynamically adjust bus routes to avoid traffic congestion, ensuring timely arrivals and improving overall efficiency.

AI-Powered Route Optimization: Machine learning algorithms can be implemented to predict the most efficient routes based on traffic patterns, historical data, and real-time factors. This would optimize bus routes and reduce travel times.

10.3 REFERENCES

[1] Haoying WU1, Sizhan ZOU1, Ning XU1, Shixu XIANG1, and Mingyu LIU, A Bus Planning Algorithm for FPC Design in Complex Scenarios, Syst., vol. 80, no. 32, pp. 3076–33775944,2024

[2], Hady W. Lauw, Topic Modelling on Document Networks With Dirichlet Optimal Transport Barycentre Delvin Ce Zhang, Syst., vol. 65, no. 70, pp. 2089–73055972 ,2024

[3] HANTAO CUI, Bus Admittance Matrix Revisited: Performance Challenges on Modern Computers ,Syst., vol. 45, no. 42, pp. 5067–46398725, 2024.

[4] Priya, S., Arun Kumar M., and Divya R. Smart Public Transport System Using Mobile Application with Flutter Framework. 2023

[5] C. Liu and L. Li, "How do subways affect urban passenger trans- port modes? —Evidence from China," Econ. Transp., vol. 23, 2020, Art. no. 100181.

[6] M. Autili, L. Chen, C. Englund, C. Pompilio, and M. Tivoli, "Cooperative intelligent transport systems: Choreography-based urban traffic coordination," IEEE Trans. Intell. Transp. Syst., vol. 22, no. 4, pp. 2088–2099, Apr. 2021.

[7] S. Selvam, A. Manisha, J. Vidhya, and S. Venkatramanan, "Fundamentals of GIS," in GIS and Geostatistical Techniques for Groundwater Science, ch. 1. New York, NY, USA: Elsevier, Cham, 2019, pp. 3–15.

[8] I. Gokasar, Y. Cetinel, and M. G. Baydogan, "Estimation of influence distance of bus stops using bus GPS data and bus stop properties," IEEE Trans. Intell. Transp. Syst., vol. 20, no. 12, pp. 4635–4642, Dec. 2019.

[9] P. Wepulanon, A. Sumalee, and W. H. K. Lam, "Temporal signa- tures of passive Wi-Fi data for estimating bus passenger waiting time at a single bus stop," IEEE Trans. Intell. Transp. Syst., vol. 21, no. 8, pp. 3366–3376, Aug. 2020.

[10] Y. Asakura, T. Iryo, Y. Nakajima, and T. Kuskabe , "Estimation of behavioural Change of railway passenger using smart card data, "public Transp., vol. 4, no. 1,pp. 1-16 ,jul 2020,doi: 10.1007/s 12469- 011-0050-0.